

✓ Tic-Tac-Toe Game with NumPy

by Cayan Kuzu 120521015

✓ Tic-Tac-Toe: An Introduction

Tic-Tac-Toe is a classic two-player game played on a 3x3 grid. Players take turns marking empty cells with their respective symbols, typically 'X' and 'O'. The objective of the game is to align three of their symbols in a row, column, or diagonal.

Rules of the Game:

1. The game is played on a 3x3 square grid.
2. The first player uses 'X', and the second player uses 'O'.
3. Players take turns placing their symbols in empty cells.
4. The player who aligns three symbols in a row, column, or diagonal wins the game.
5. If all cells are filled and no player has won, the game ends in a **draw**.

Strategy:

Although the rules are simple, Tic-Tac-Toe requires players to make thoughtful moves. To increase their chances of winning, players must simultaneously work on completing their own rows while blocking their opponent's attempts. This balance makes the game both fun and strategic.

✓ Roadmap and code

This guide will help you step by step to create a Tic-Tac-Toe game. It focuses on simple logic and using Python's NumPy library for board operations.

✓ 1. Plan the Game

- **Main requirements:**

- A game board (e.g., 3x3 size or adjustable).
- Two players (X and O).
- Rules for winning (3 in a row, column, or diagonal).

- **Classes to create:**

- **Game:** Controls the game flow.
- **Board:** Handles the game board and its logic.
- **Player:** Stores player names and symbols.

- **Why use NumPy?**

- NumPy makes it easy to work with a matrix (game board).
- You can use it for efficient checks like finding a winner or checking if the board is full.

-- !!! AND for first code line *DO NOT FORGET TO IMPORT NUMPY LIBRARY*

```
1 import numpy as np
```

✓ 2. Build the Board Class

- **Set up the board:**

- Use NumPy to create an empty board (e.g., `np.zeros` or `np.full`).
- Allow the board size to be set when the game starts.

- **Add board functions:**

- `place_symbol(row, col, symbol):`
 - Put a symbol (X or O) in the selected cell.
 - Check if the move is valid (cell must be empty).
- `is_full():`
 - Check if the board is completely filled.
- `check_winner():`
 - Check for 3 in a row, column, or diagonal.
 - Use NumPy functions like `np.diagonal` or `np.all` for this.
- `display_board():`
 - Show the board in a clean way in the terminal.

```
1 class Board:
2     def __init__(self, size):
3         self.board_size = size # size will be 3 which is size when we will creating ga
```

```

4     self.board = np.zeros((size, size), dtype=int) #creating zero matrix with givn
5 def place_symbol(self, row, col, symbol):
6     if row < 0 or row >= self.board_size or col < 0 or col >= self.board_size: #ch
7         print("Invalid move: Choose a cell within the board's boundaries.")
8         return False
9     if self.board[row, col] != 0:
10        print("Invalid move: This cell is already occupied.") #ve create matrix wi
11        return False
12    if symbol == 'X':
13        self.board[row, col] = 1 #puttig 1 into cell which is player wants to put
14    elif symbol == 'O':
15        self.board[row, col] = -1 #puttig -1 into cell which is player wants to pu
16    return True
17 def display_board(self):
18     for row in self.board:
19         for cell in row:
20             if cell == 0:
21                 print(" ", end="|")
22             elif cell == 1:
23                 print("X", end="|")
24             else:
25                 print("O", end="|")
26         print("\n" + "-" * (2 * self.board_size - 1))
27     print("\n")
28 def is_full(self):
29     return np.all(self.board != 0) #checking which cells is 0. this give us ture f
30 def check_winner(self):
31     for row in range(self.board_size): #self.board_size equal 3
32         row_sum = np.sum(self.board[row, :]) #for exemple. takeing 1. row and sum
33         if row_sum == self.board_size: #if this sumation equals board_size which i
34             return 1 #then return 1. so this means X wins. because X presents 1
35         if row_sum == -self.board_size: #if this sumation equals -(board_size) wh
36             return -1 #then return -1. so this means O wins. because O presents -
37     for col in range(self.board_size): #same operations for cols
38         col_sum = np.sum(self.board[:, col]) #same operations for cols
39         if col_sum == self.board_size: #same operations for cols
40             return 1 #same operations for cols
41         if col_sum == -self.board_size: #same operations for cols
42             return -1 #same operations for cols
43     main_diag_sum = np.sum(np.diagonal(self.board)) #same operations for diagonals
44     if main_diag_sum == self.board_size: #same operations for diagonal
45         return 1 #same operations for diagonal
46     if main_diag_sum == -self.board_size: #same operations for diagonal
47         return -1 #same operations for diagonal
48     anti_diag_sum = np.sum(np.diagonal(np.fliplr(self.board))) #same operations fo
49     if anti_diag_sum == self.board_size: #same operations for reverse diagonal
50         return 1 #same operations for reverse diagonal
51     if anti_diag_sum == -self.board_size: #same operations for reverse diagonal
52         return -1 #same operations for reverse diagonal
53     return 0 #if there is no winer return 0

```

Summary: We create a 3x3 game table using a matrix. In this implementation, we can check who is the winner and whether the table is full.

- An important part is that we use numbers for X and O symbols. When checking the winner, we utilize matrix calculations. In column and row checks, we perform a basic sum operation on the matrix. The main idea behind this is: if any row or column is filled with X (whose numerical value is 1), the sum of this column or row must be 3, which is equal to our table size. Similarly, if it is filled with O (-1), their sum becomes -3, which is the negative value of our table size.
- Another important part is checking the diagonals, which include the main diagonal and the anti-diagonal. For the main diagonal, we can simply use numpy's diagonal summation. On the other hand, when checking the anti-diagonal, we can use the **fliplr()** function from numpy. The `np.fliplr()` function flips (inverts) a matrix horizontally, meaning the order of the matrix's columns is reversed. For example, check this: ``

```

1 import numpy as np
2
3 array = np.array([[1, 2, 3],
4                  [4, 5, 6],
5                  [7, 8, 9]])
6
7 flipped = np.fliplr(array)
8 print(flipped)
9

```

```

→ [[3 2 1]
   [6 5 4]
   [9 8 7]]

```

✓ 3. Create the Player Class

- **Store player information:**
 - `name`: The name of the player.
 - `symbol`: The player's symbol (X or O).
- **Keep it simple:**
 - This class only manages player info and switching turns.

```

1 class Player:
2     def __init__(self, name, symbol):
3         self.name = name
4         self.symbol = symbol

```

✓ 4. Create the Game Class

- **Add functions for gameplay:**

- `start_game()`:
 - Begin the game and control the rounds.
- `play_turn()`:
 - Let players take turns to play.
- `switch_player()`:
 - Alternate between Player X and Player O.
- `check_winner()`:
 - Call the `check_winner()` function from the Board class.
- Check for a draw:
 - If `is_full()` is true and no winner is found, the game ends in a draw.
- **Ask for player input:**
 - Get the row and column number where the player wants to place their symbol.
 - Make sure the input is valid (within bounds and not already taken).
- **Handle invalid moves:**
 - Show an error message if the move is invalid.
 - Let the player try again.

```

1 class Game:
2     def __init__(self, size): #this size == 3 (which we give while starting game)
3         self.board = Board(size) # creating board object from Board class with (size)x
4         self.player1 = Player("Player 1", "X") #creating player1 from Player class
5         self.player2 = Player("Player 2", "O") #creating player2 from Player class
6         self.current_player = self.player1 # setting player who will first play when g
7     def start_game(self):
8         while True:
9             self.board.display_board() #show the board from Board class (because self
10            print(f"\n{self.current_player.name}'s ({self.current_player.symbol}) Turn
11            try:
12                row = int(input("Row (1-3): ")) #getting row's number which is will fi
13                col = int(input("Column (1-3): ")) #getting row's number which is will
14            except ValueError:
15                print("Invalid input, please enter a number.")
16                continue
17            if self.board.place_symbol(row-1, col-1, self.current_player.symbol): #pla
18                winner = self.board.check_winner() # checking if there is a winner qui
19                if winner != 0: #this means every time after the check,
20                    self.board.display_board() #show board
21                    if winner == 1: #this means if winner is x. because from self.boar
22                        print(f"Winner: {self.player1.name} (X)")
23                    else:
24                        print(f"Winner: {self.player2.name} (O)") #this means winner i
25                    break
26                if self.board.is_full(): #this means there is no another free cell.
27                    self.board.display_board() #show board

```

```

28         print("It's a draw!") #its draw becaus no winner
29         break
30     self.switch_player() #its really whitching player while game is on
31 def switch_player(self):
32     if self.current_player == self.player1: #setting current player to player1
33         self.current_player = self.player2 #ad quick after, setting current player
34     else:
35         self.current_player = self.player1 #if current player is player 2, then se

```

Form here we use while loop because we want to our game on while some player win or draw. The logic of this while loop is like Pygame.

✓ 5. Final Steps

- Sart the game (:

```

1 game = Game(3) #creating game from Game class with set size of board
2 game.start_game() #starting game from Game class

```

✓ Exemple round and full code

✓ Full Code

```

1 # -*- coding: utf-8 -*-
2 import numpy as np
3
4 class Board:
5     def __init__(self, size):
6         self.board_size = size # size is accualy 3 which is size when we creating game
7         self.board = np.zeros((size, size), dtype=int) #creating zero matrix with givn
8     def place_symbol(self, row, col, symbol):
9         if row < 0 or row >= self.board_size or col < 0 or col >= self.board_size: #ch
10             print("Invalid move: Choose a cell within the board's boundaries.")
11             return False
12         if self.board[row, col] != 0:
13             print("Invalid move: This cell is already occupied.") #ve create matrix wi
14             return False
15         if symbol == 'X':
16             self.board[row, col] = 1 #puttig 1 into cell which is player wants to put
17         elif symbol == 'O':
18             self.board[row, col] = -1 #puttig -1 into cell which is player wants to pu
19         return True
20     def display_board(self):
21         for row in self.board:

```

```

22         for cell in row:
23             if cell == 0:
24                 print(" ", end="|")
25             elif cell == 1:
26                 print("X", end="|")
27             else:
28                 print("O", end="|")
29         print("\n" + "-" * (2 * self.board_size - 1))
30     print("\n")
31     def is_full(self):
32         return np.all(self.board != 0) #checking which cells is not 0. this give us tu
33     def check_winner(self):
34         for row in range(self.board_size): #self.board_size equal 3
35             row_sum = np.sum(self.board[row, :]) #for exemple. takeing 1. row and sum
36             if row_sum == self.board_size: #if this sumation equals board_size which i
37                 return 1 #then return 1. so this means X wins. because X presents 1
38             if row_sum == -self.board_size: #if this sumation equals -(board_size) wh
39                 return -1 #then return -1. so this means O wins. because O presents -
40         for col in range(self.board_size): #same operations for cols
41             col_sum = np.sum(self.board[:, col]) #same operations for cols
42             if col_sum == self.board_size: #same operations for cols
43                 return 1 #same operations for cols
44             if col_sum == -self.board_size: #same operations for cols
45                 return -1 #same operations for cols
46         main_diag_sum = np.sum(np.diagonal(self.board)) #same operations for diagonals
47         if main_diag_sum == self.board_size: #same operations for diagonal
48             return 1 #same operations for diagonal
49         if main_diag_sum == -self.board_size: #same operations for diagonal
50             return -1 #same operations for diagonal
51         anti_diag_sum = np.sum(np.diagonal(np.fliplr(self.board))) #same operations fo
52         if anti_diag_sum == self.board_size: #same operations for reverse diagonal
53             return 1 #same operations for reverse diagonal
54         if anti_diag_sum == -self.board_size: #same operations for reverse diagonal
55             return -1 #same operations for reverse diagonal
56         return 0 #if there is no winer return 0
57
58     class Player:
59         def __init__(self, name, symbol):
60             self.name = name
61             self.symbol = symbol
62
63     class Game:
64         def __init__(self, size): #this size == 3 (which we give while starting game)
65             self.board = Board(size) # creating board object from Board class with (size)x
66             self.player1 = Player("Player 1", "X") #creating player1 from Player class
67             self.player2 = Player("Player 2", "O") #creating player2 from Player class
68             self.current_player = self.player1 # setting player who will first play when g
69         def start_game(self):
70             while True:
71                 self.board.display_board() #show the board from Board class (because self
72                 print(f"\n{self.current_player.name}'s ({self.current_player.symbol}) Turn
73                 try:
74                     row = int(input("Row (1-3): ")) #getting row's number which is will fi
75                     col = int(input("Column (1-3): ")) #getting colum's number which is wi
76                     except ValueError:

```

```

77         print("Invalid input, please enter a number.")
78         continue
79     if self.board.place_symbol(row-1, col-1, self.current_player.symbol): #pla
80         winner = self.board.check_winner() # checking if there is a winner qui
81         if winner != 0: #this means every time after the check,
82             self.board.display_board() #show board
83             if winner == 1: #this means if winner is x. because from self.boar
84                 print(f"Winner: {self.player1.name} (X)")
85             else:
86                 print(f"Winner: {self.player2.name} (O)") #this means winner i
87             break
88         if self.board.is_full(): #this means there is no another free cell.
89             self.board.display_board() #show board
90             print("It's a draw!") #its draw becaus no winner
91             break
92         self.switch_player() #its really whitching player while game is on
93     def switch_player(self):
94         if self.current_player == self.player1: #setting current player to player1
95             self.current_player = self.player2 #ad quick after, setting current player
96         else:2
97             self.current_player = self.player1 #if current player is player 2, then se
98
99 # Start the game
100 game = Game(3) #creating game from Game class with set size of board
101 game.start_game() #starting game from Game class

```

Exemple

```

| | | |

```

```

| | | |

```

```

| | | |

```

Player 1's (X) Turn

Row (1-3): 1

Column (1-3): 1

```

|X| | |

```

```

| | | |

```

```

| | | |

```

Player 2's (O) Turn

Row (1-3): 2

Column (1-3): 3

```
|X| | |
```

```
| | |O|
```

```
| | | |
```

Player 1's (X) Turn

Row (1-3): 1

Column (1-3):

Invalid input, please enter a number.

```
|X| | |
```

```
| | |O|
```

```
| | | |
```

Player 1's (X) Turn

Row (1-3): 2

Column (1-3): 2

```
|X| | |
```

```
| |X|O|
```

```
| | | |
```

Player 2's (O) Turn

Row (1-3): 3

Column (1-3): 1

```
|X| | |
```

```
| |X|O|
```

```
|O| | |
```

Player 1's (X) Turn

Row (1-3): 3

Column (1-3): 3

|X| | |

| |X|O|

|O| |X|

Winner: Player 1 (X)

✓ Referances

- <https://stackoverflow.com/questions/43546307/python-oop-tic-tac-toe>
- <https://www.geeksforgeeks.org/python-numpy/>
- https://www.youtube.com/watch?v=Qdbcqw2KX_Q
- https://www.youtube.com/watch?v=K8DvLN_Pp_s
- ChatGPT (NOT used full time and NOT on all the code. Usually fixing some bugs, checking and understanding other code (e.g. uplinks), creating a more analytical roadmap, etc.)